

User application guideline

Step	Action	Description	Expected Outcome	Page/Redirect
1	Access the Registration Page	Navigate to the registration page (usually /register) to create a new user account.	Displays the registration form.	/register
2	Fill in Registration Form	Enter required details (such as username, email, password) into the form.	Form data is collected and validated.	Form fields are populated.
3	Submit Registration Form	Submit the form by clicking the Submit or Register button.	The form is validated. If valid, a new user is created.	Redirects to the "ping_operation" page after successful registration.
4	Successful Registration	After form submission, the system registers the new user and logs them in automatically.	The user is logged in, starting a session.	Redirected to the "ping_operation" page.
5	Access the Login Page	Navigate to the login page (usually /login) if the user already has an account.	Displays the login form.	/login
6	Fill in Login Credentials	Enter username and password into the login form.	Form data is entered for login authentication.	Form fields are populated.
7	Submit Login Form	Click the Login button after entering credentials.	The system attempts to authenticate the user.	Redirects based on successful authentication or failure.

8	Authentication Success	If the credentials are correct, the user is authenticated and logged in.	A session is created for the authenticated user.	Redirected to the page specified in the "next" URL parameter (defaults to "ping_operation" if no "next" is specified).
9	Authentication Failure	If the credentials are incorrect, the user is not authenticated.	The system will display an error message or re-render the login form.	Stays on the login page with error feedback.
10	Accessing Protected Pages	After successful login, access other parts of the application (e.g., "ping_operation" page) as an authenticated user.	User has access to protected resources.	"ping_operation" or other secured pages.

Key Notes

- **User Registration:** When a new user is registered, they are automatically logged in and redirected to the dashboard ([ping_operation](#) page).
- **Login Process:** If the user already exists, they can log in using their credentials. The login page may redirect to the original page the user tried to access before being prompted to log in (through the [next](#) URL).
- **Error Handling:** In case of incorrect login credentials, the user remains on the login page with an error message.

Ping Operation Application Guideline

Operation	Description	Result
Enable Ping	A ping test is sent to check if the target device or host is reachable over the network.	- Device is alive: The device responds to the ping, indicating it is reachable. - Device is unreachable: The device does not respond, indicating it is either offline or there is a network issue.

DNS application guideline

Below is a detailed table format guideline for the DNS management application based on the provided Django views:

Feature	Description	Implementation Details	Expected Outcome
DNS List View	Displays a list of saved DNS records on the <code>dns.html</code> page.	- Query the database to get DNS records using <code>DNS.objects.values("dns_name", "dns_created_at")</code> .	- The page will show all DNS names and their creation date if any DNS records exist. - If no records exist, it will show an empty page.
Add DNS	Allows users to add a new DNS record.	- Retrieve the DNS address from <code>request.POST.get("dns_address")</code> . - Create a new DNS record using <code>DNS.objects.create(dns_name=get_dns_id)</code> .	- A new DNS record is created with the provided DNS address and added to the database. - The page redirects to the DNS list view.
Delete DNS	Allows users to delete a DNS record based on its unique ID.	- Fetch the DNS object using <code>get_object_or_404(DNS, dns_name=dns_id)</code> . - Delete the object using <code>dns.delete()</code> .	- The DNS record corresponding to the provided DNS name is deleted from the database. - The page redirects to the DNS list view.

- **Function:** Deletes a DNS record identified by `dns_id` passed via URL.
- **Action:**
 - Fetch the DNS object by `dns_name` using `get_object_or_404(DNS, dns_name=dns_id)`.
 - Deletes the object using `dns.delete()`.
- **Outcome:** Redirects to the DNS list view after deletion. If the DNS record is not found, a 404 error will be raised.

Error Handling:

- **Deleting Non-Existent DNS:** The `get_object_or_404()` function ensures that the application handles cases where the DNS record does not exist. This prevents server errors and displays a 404 page instead.

Logging:

Log Entries: The logging mechanism will track DNS creation and deletion operations. For example:

```
logger.info(f"DNS {get_dns_id} added.")
logger.info(f"DNS {dns_id} deleted.")
```

- These logs will be helpful for debugging and keeping track of DNS actions.